Steven An

Dec 10, 2025

ECE4574

Drug-Interaction Checker: Project Final Report

Summary

What does it do?

The Drug Interaction Checker is a role-based web application designed to improve medication safety, enhance communication between patients and healthcare providers, and simplify prescription management. The system provides three user roles Patient, Doctor, and Administrator with each with specific functionality accessed through a front-end web portal.

How is it used?

Users log in through a web browser and are routed to different portal interfaces depending on their role Patient, Doctor, or Admin. Each portal provides access to role-specific features via a simple, intuitive interface. Doctors manage clinical tasks, patients view personalized information, and admins oversee system user and data integrity. All data is stored securely and delivered through Superbase's authenticated REST API.

What are the benefits?

The system improves patient communication and safety by automatically identifying potentially dangerous drug interactions and generating clear, patient-friendly medication warnings. It streamlines clinical workflows by giving doctors a centralized place to manage patients, prescriptions, and clinical notes. Because the platform relies on FDA drug label data and secure role-based access controls, it strengthens decision-making while maintaining secure data. Finally, the lightweight architecture built on a front-end client and Supabase backend keeps the system scalable, fast, and easy to maintain.

My Reflection

We followed an iterative, incremental development process using Scrum based development. Early milestones focused on establishing authentication, database schema, and user roles. Later iterations added medication logic, interaction checking, UI improvements, and safety features. We routinely tested integrations as features were added, which helped catch issues early particularly with Supabase policies and openFDA API inconsistencies.

Overall, the project not only demonstrated our technical capability to design and deploy a real application, but also strengthened our teamwork, adaptability, and understanding of integrating third-party APIs with secure cloud services. The experience has prepared us well for future full-stack and health-tech-related development work.

Appendix

styles.css

```
/*
   styles.css     Virginia Tech      Sprint 1,2,3
   Main CSS stylesheet for the Drug Interaction Checker frontend.
   Provides global styles, layout, and component styling for a clean and consistent UI.
*/

@import
url('https://fonts.googleapis.com/css2?family=IBM+Plex+Mono:wght@400;600&display=swap')
;

/* Global Reset */
* {
   box-sizing: border-box;
   margin: 0;
   padding: 0;
   font-family: "IBM Plex Mono", monospace;
   color: #000;
}

/* Page Layout */
body {
   background: #eee7e2;
   min-height: 100vh;
   margin: 0;
```

```
}

/* Container Card */
.container {
    background: #eee7e2;
    border-radius: 0;
    box-shadow: none;
    width: 100%;
    max-width: none;
    padding: 16px;
    text-align: left;
    color: #000;
}

.topbar {
    position: sticky;
    top: 0;
    width: 100%;
    display: flex;
    align-items: center;
    justify-content: flex-end;
    gap: 12px;
    padding: 12px 16px;
    background: #eee7e2;
    border-bottom: 1px solid #000;
```

```
}

.topbar .role { color: #000; }

/* Everett Miceli Simple tabs for topbar */
.topbar .tab {
  background: #eee7e2;
  color: #000;
  border: 1px solid #000;
  padding: 0.5rem 0.9rem;
}
.topbar .tab.active {
  background: #000;
  color: #eee7e2;
}

.page {
  padding: 16px;
  max-width: 1200px;
  margin: 0 auto;
}

/* Headings */
h1, h2 {
  color: #2b2b2b;
```

```
    margin-bottom: 1rem;
}

h3 { margin: 0 0 0.75rem 0; }

/* Paragraphs */
p {
    color: #2b2b2b;
    margin: 0.75rem 0 1.25rem 0;
}

/* Form Styling */
form {
    display: flex;
    flex-direction: column;
    gap: 0.9rem;
    text-align: left;
}

label {
    font-weight: 600;
    color: #2b2b2b;
}

input, select {
```

```
    padding: 0.7rem;
    border: 1.5px solid #000;
    border-radius: 0;
    font-size: 1rem;
    transition: 0.2s;
    background: #eee7e2;
    color: #000;
}

input:focus, select:focus {
    border-color: #000;
    outline: none;
}

/* Buttons */
button {
    background-color: #eee7e2;
    color: #000;
    border: 1px solid #000;
    padding: 0.8rem 1.2rem;
    border-radius: 0;
    font-weight: 600;
    cursor: pointer;
    transition: 0.2s;
}
```

```
button:hover {
    background-color: #f2f2f2;
}

/* Secondary Buttons */
button.secondary {
    background-color: #eee7e2;
    color: #000;
    border: 1px solid #000;
}

button.secondary:hover {
    background-color: #f2f2f2;
}

/* Special style for Logout button */
.logout-btn {
    background: #000;
    color: #eee7e2;
    border: 1px solid #000;
}
.logout-btn:hover { background: #111; }

/* Panels, Lists, Grid */
```

```
.panel {
  border: 1px solid #000;
  padding: 12px;
  border-radius: 0;
  background: #eee7e2;
}

.grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

.list { list-style: none; padding: 0; margin: 0; }
.list li {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 6px 0;
  border-bottom: 1px dashed #000;
}
.list li:last-child { border-bottom: none; }

.row { margin: 8px 0; }
.muted { color: #444; font-size: 12px; }
```

```
.rx-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(260px, 1fr)); gap: 16px; }

.card { border: 1px solid #000; background: #eee7e2; padding: 12px; }


.small-btn { padding: 0.25rem 0.5rem; font-size: 0.85rem; }


/* Responsive */

@media (max-width: 480px) {

   .container {

      padding: 1.5rem;

   }


   h1, h2 {

      font-size: 1.4rem;

   }

   .grid { grid-template-columns: 1fr; }

}


/* Sprint 2, Steven An */

/* Interaction Checker Box */

.interaction-panel {

  margin-top: 10px;

  padding: 10px;

  max-width: 100%;

  word-wrap: break-word;
```

```
  white-space: normal;
}

/* Sprint 3, Steven An */
/* Individual interaction items */
.interaction-item {
  border-bottom: 1px dashed #999;
  padding-bottom: 10px;
  margin-bottom: 10px;
}

/* Sprint 2, Steven An */
/* Show More button */
.toggle-btn {
  margin-top: 5px;
  padding: 4px 8px;
  background: #eee7e2;
  border: 1px solid #000;
  cursor: pointer;
}

.toggle-btn:hover {
  background: #eaeaea;
}
```

```
.warning-card {
  border-left: 4px solid #d97757;
  padding: 8px 12px;
  margin-bottom: 8px;
  background: #fff7ed;
  border-radius: 6px;
}

.tiny-muted {
  font-size: 0.8rem;
  opacity: 0.7;
  margin-top: 8px;
}

/* Everett Miceli Home/Logo button in top left */
.app-logo {
    position: fixed;
    top: 12px;
    left: 12px;
    width: 36px;
    height: 36px;
    z-index: 1000;
    cursor: pointer;
    user-select: none;
    object-fit: contain;
```

```
}

#patientSearch { width: 100%; }

/* Everett Miceli Offset logo on create account page */
.logo-offset { padding-left: 60px; }

/* Everett Miceli Delete Button for Admin Page (in individual accountview) */
#deleteAccountBtn {
   background: #eee7e2;
   color: #b00020;
   border-color: #b00020;
}
#deleteAccountBtn:hover {
   background: #ff6e6e;
}

/* Everett Miceli Delete button for Admin List */
.danger-btn,
.remove-btn {
   background: #eee7e2;
   color: #b00020;
   border-color: #b00020;
}
.danger-btn:hover,
```

```
.remove-btn:hover {
    background: #ff6e6e;
}

/* Everett Miceli Alignment changes for doctor page medication fields */
#prescriptionForm .row > label {
    display: inline-block;
    min-width: 120px;
}
#prescriptionForm .row > input,
#prescriptionForm .row > select {
    margin-left: 12px;
}
#rxMedicationName {
    width: 520px;
    max-width: 95%;
}
#interactionInput {
    width: 420px;
    max-width: 95%;
    margin-right: 12px;
}
```

admin.js

```
// Everett Miceli
/*
  admin.js     Virginia Tech      Sprint 1,2,3
  Admin page modified from doctor.js that allows for management of user accunts and role assignment

  Allows administrators to delete user accounts, reassign roles, view email, userid, and name for recovery purposes
  Also allows for ability to act as any role (patient or doctor) for development purposes
*/
const logoutBtn = document.getElementById('logoutBtn');
const headerRole = document.getElementById('headerRole');
const myPatientsList = document.getElementById('myPatientsList');
const patientSearch = document.getElementById('patientSearch');
const patientDetails = document.getElementById('patientDetails');
const detailTitle = document.getElementById('detailTitle');
const detailsPanel = document.getElementById('detailsPanel');
const adminRoleSelect = document.getElementById('adminRoleSelect');
const saveRoleBtn = document.getElementById('saveRoleBtn');
const deleteAccountBtn = document.getElementById('deleteAccountBtn');

let selectedAccount = null;

init();
```

```
async function init() {
  const token = getAccessToken();
  const email = getEmail();
  if (!token || !email) { window.location.href = 'home.html'; return; }

  logoutBtn.addEventListener('click', () => { clearSession(); window.location.href =
'home.html'; });
  patientSearch.addEventListener('input', renderMyPatientsFiltered);
  saveRoleBtn.addEventListener('click', onSaveRole);
  deleteAccountBtn.addEventListener('click', onDeleteAccount);

  // Admin only, deny access to those without admin role
  const rows = await supabaseRequest({ method: 'GET', path: 'user_accounts', accessToken:
token, query: `?select=userid,fullname,role&email=eq.${encodeURIComponent(email)}` });
  if (!rows || rows.length === 0) { alert('No profile found'); return; }
  const me = rows[0];
  const isAdmin = (me.role || '').toLowerCase() === 'administrator';
  if (!isAdmin) { alert('Administrator access required.'); window.location.href = 'home.html';
return; }
  headerRole.textContent = `Admin: ${me.fullname || email}`;

  await refreshLists();
}

async function refreshLists() {
  const token = getAccessToken();
```

```
  const users = await supabaseRequest({ method: 'GET', path: 'user_accounts', accessToken: token, query: `?select=userid,fullname,email,role&order=fullname.asc` });
  window.__myPatients = users;
  renderMyPatientsFiltered();
}

function renderMyPatientsFiltered() {
  const q = (patientSearch.value || '').toLowerCase();
  const list = (window.__myPatients || []).filter(p => {
    const t = `${p.fullname || ''} ${p.email || ''} ${p.userid || ''}`.toLowerCase();
    return t.includes(q);
  });
  myPatientsList.innerHTML = '';
  list.forEach(p => {
    const li = document.createElement('li');
    li.style.justifyContent = 'flex-start';
    li.style.gap = '8px';
    const label = document.createElement('span');
    label.textContent = `${p.fullname || p.email} (${p.role || 'patient'})`;
    const open = document.createElement('button'); open.textContent = 'View'; open.addEventListener('click', () => openPatient(p));
    const remove = document.createElement('button'); remove.textContent = 'Delete'; remove.className = 'small-btn danger-btn'; remove.addEventListener('click', () => deleteAccount(p.userid));
    li.appendChild(label);
    const actions = document.createElement('div');
    actions.style.marginLeft = 'auto';
```

```
    actions.style.display = 'flex';
    actions.style.gap = '8px';
    actions.appendChild(open);
    actions.appendChild(remove);
    li.appendChild(actions);
    myPatientsList.appendChild(li);
  });
}

function clearDetails() {
  detailTitle.textContent = 'Account Details';
  patientDetails.textContent = 'Select an account to view details.';
  detailsPanel.style.display = 'none';
}

function openPatient(patient) {
  selectedAccount = patient;
  document.getElementById('overviewSection').style.display = 'none';
  detailsPanel.style.display = 'block';
  detailTitle.textContent = `Account — ${(patient.fullname || patient.email || patient.userid)}`;
  patientDetails.textContent = `${patient.userid} • ${patient.email}`;
  //set role
  const current = (patient.role || '').toLowerCase();
  adminRoleSelect.value = current === 'administrator' ? 'administrator' : (patient.role ||
'patient');
}
```

```
async function onSaveRole() {
  if (!selectedAccount) return;
  const token = getAccessToken();
  const newRole = adminRoleSelect.value;
  await supabaseRequest({ method: 'PATCH', path: 'user_accounts', accessToken: token, query: `?userid=eq.${encodeURIComponent(selectedAccount.userid)}`, body: { role: newRole } });
  await refreshLists();
}

async function onDeleteAccount() {
  if (!selectedAccount) return;
  await deleteAccount(selectedAccount.userid);
}

async function deleteAccount(userid) {
  const token = getAccessToken();
  if (!confirm(`Delete account ${userid}? This cannot be undone.`)) return;
  await supabaseRequest({ method: 'DELETE', path: 'user_accounts', accessToken: token, query: `?userid=eq.${encodeURIComponent(userid)}` });
  selectedAccount = null;
  clearDetails();
  document.getElementById('overviewSection').style.display = 'grid';
  await refreshLists();
}
```

doctor.js

```
/*
  doctor.js     Virginia Tech      Sprint 1,2,3
  Doctor portal, this file handle all doctor-related frontend logic and UI

  Allows doctors to assign patients, view patient details, add notes, and prescribe medications.
  Also includes FDA drug interaction checking functionality (making sure two drugs do not have harmful interactions),
  and questionnaire response viewing
*/

// Sprint 2, Steven An
// Helpers functions for FDA interaction UI
function truncate(text, maxLength = 150) {
  if (!text) return "";
  if (text.length <= maxLength) return text;
  return text.slice(0, maxLength) + "...";
}

function toggleInteraction(btn) {
  const container = btn.closest(".interaction-item");
  const shortText = container.querySelector(".short-text");
  const fullText = container.querySelector(".full-text");

  if (fullText.style.display === "none") {
    fullText.style.display = "inline";
```

```
    shortText.style.display = "none";
    btn.textContent = "Show Less";
  } else {
    fullText.style.display = "none";
    shortText.style.display = "inline";
    btn.textContent = "Show More";
  }
}

//  DOM elements
const logoutBtn       = document.getElementById("logoutBtn");
const headerRole      = document.getElementById("headerRole");
const unassignedList  = document.getElementById("unassignedList");
const myPatientsList  = document.getElementById("myPatientsList");
const patientSearch   = document.getElementById("patientSearch");
const detailTitle     = document.getElementById("detailTitle");
const patientDetails  = document.getElementById("patientDetails");
const detailsPanel    = document.getElementById("detailsPanel");

// Medication form
const rxMedicationName = document.getElementById("rxMedicationName");
const rxDosage         = document.getElementById("rxDosage");
const rxForm           = document.getElementById("rxForm");
const rxNotes          = document.getElementById("rxNotes");
const rxStart          = document.getElementById("rxStart");
```

```
const rxEnd           = document.getElementById("rxEnd");
const rxSave          = document.getElementById("rxSave");

// Notes
const notesList  = document.getElementById("notesList");
const newNote    = document.getElementById("newNote");
const addNoteBtn = document.getElementById("addNoteBtn");

// Med list
const rxList = document.getElementById("rxList");

// Globals
let doctorUserId    = null;
let selectedPatient = null;
let myPatients      = [];
let medicationsById = {};

//  Init
init();

async function init() {
  const token = getAccessToken();
  const email = getEmail();
  if (!token || !email) {
    window.location.href = "home.html";
```

```
  return;
}

if (logoutBtn) {
  logoutBtn.addEventListener("click", () => {
    clearSession();
    window.location.href = "home.html";
  });
}

if (patientSearch) {
  patientSearch.addEventListener("input", renderMyPatientsFiltered);
}
if (rxSave) {
  rxSave.addEventListener("click", onSavePrescription);
}
if (addNoteBtn) {
  addNoteBtn.addEventListener("click", onAddNote);
}

const interactionBtn = document.getElementById("interactionBtn");
if (interactionBtn) {
  interactionBtn.addEventListener("click", handleInteractionCheck);
}
```

```
// Load doctor profile
const meRows = await supabaseRequest({
  method: "GET",
  path: "user_accounts",
  accessToken: token,
  query:
`?select=userid,fullname,role,email&email=eq.${encodeURIComponent(email)}&limit=1`
});

if (!meRows || meRows.length === 0) {
  alert("No profile found");
  return;
}

const me = meRows[0];
doctorUserId = me.userid;

if (headerRole) {
  headerRole.textContent = `Doctor: ${me.fullname || me.email || ""}`;
}

// Cache medications for display
const medRows = await supabaseRequest({
  method: "GET",
  path: "medications",
  accessToken: token,
```

```
    query: "?select=id,name&order=name.asc"
  });

  medicationsById = {};
  (medRows || []).forEach(m => {
    medicationsById[m.id] = m.name;
  });

  await refreshLists();
}

//  Patient list / assignments
async function refreshLists() {
  const token = getAccessToken();

  // All patients
  const patients = await supabaseRequest({
    method: "GET",
    path: "user_accounts",
    accessToken: token,
    query: "?select=userid,fullname,email,role&role=eq.patient&order=fullname.asc"
  });

  // All assignments
  const assignments = await supabaseRequest({
```

```
    method: "GET",
    path: "patient_doctor_assignments",
    accessToken: token,
    query: "?select=patient_userid,doctor_userid,notes"
  });

  const assignedSet = new Set((assignments || []).map(a => a.patient_userid));

  const unassigned = (patients || []).filter(p => !assignedSet.has(p.userid));
  renderUnassigned(unassigned);

  // My patients = those assigned to THIS doctor
  const myIds = new Set(
    (assignments || [])
      .filter(a => a.doctor_userid === doctorUserId)
      .map(a => a.patient_userid)
  );
  myPatients = (patients || []).filter(p => myIds.has(p.userid));

  renderMyPatientsFiltered();
}

function renderUnassigned(list) {
  if (!unassignedList) return;
  unassignedList.innerHTML = "";
```

```
  list.forEach(p => {
    const li = document.createElement("li");

    const nameSpan = document.createElement("span");
    nameSpan.textContent = p.fullname || p.email || p.userid;

    const btn = document.createElement("button");
    btn.textContent = "Assign to me";
    btn.addEventListener("click", () => assignPatient(p.userid));

    li.appendChild(nameSpan);
    li.appendChild(btn);
    unassignedList.appendChild(li);
  });
}

function renderMyPatientsFiltered() {
  if (!myPatientsList) return;
  myPatientsList.innerHTML = "";
  const q = (patientSearch && patientSearch.value ? patientSearch.value : "").toLowerCase();

  const filtered = myPatients.filter(p => {
    const name  = (p.fullname || "").toLowerCase();
    const email = (p.email || "").toLowerCase();
    return name.includes(q) || email.includes(q);
```

```
  });

  filtered.forEach(p => {
    const li = document.createElement("li");

    const label = document.createElement("span");
    label.textContent = p.fullname || p.email || p.userid;

    const viewBtn = document.createElement("button");
    viewBtn.textContent = "View";
    viewBtn.addEventListener("click", () => openPatient(p));

    const removeBtn = document.createElement("button");
    removeBtn.className = "small-btn danger-btn remove-btn";
    removeBtn.style.color = "#b00020";
    removeBtn.style.borderColor = "#b00020";
    removeBtn.textContent = "Remove";
    removeBtn.addEventListener("click", () => unassignPatient(p.userid));

    li.appendChild(label);
    li.appendChild(viewBtn);
    li.appendChild(removeBtn);
    myPatientsList.appendChild(li);
  });
}
```

```
async function assignPatient(patientUserId) {
  const token = getAccessToken();

  await supabaseRequest({
    method: "POST",
    path: "patient_doctor_assignments",
    accessToken: token,
    body: [{
      patient_userid: patientUserId,
      doctor_userid: doctorUserId
    }]
  });

  await refreshLists();
}

async function unassignPatient(patientUserId) {
  const token = getAccessToken();

  await supabaseRequest({
    method: "DELETE",
    path: "patient_doctor_assignments",
    accessToken: token,
```

```
    query:
`?patient_userid=eq.${encodeURIComponent(patientUserId)}&doctor_userid=eq.${encodeURIC
omponent(doctorUserId)}`
  });

  if (selectedPatient && selectedPatient.userid === patientUserId) {
    selectedPatient = null;
    clearDetails();
  }

  await refreshLists();
}

//  Patient details
function clearDetails() {
  if (detailTitle) {
    detailTitle.textContent = "Patient Details";
  }
  if (patientDetails) {
    patientDetails.textContent = "Select a patient to view notes and medications.";
  }
  if (detailsPanel) {
    detailsPanel.style.display = "none";
  }
  if (rxList) rxList.innerHTML = "";
  if (notesList) notesList.innerHTML = "";
```

```
  const formPanel = document.getElementById("prescriptionForm");
  if (formPanel) formPanel.style.display = "none";
}

async function openPatient(patient) {
  selectedPatient = patient;

  if (detailTitle) {
    detailTitle.textContent = `Patient Details — ${patient.fullname || patient.email || ""}`;
  }
  if (patientDetails) {
    patientDetails.textContent = `${patient.userid} • ${patient.email || ""}`;
  }

  if (detailsPanel) {
    detailsPanel.style.display = "block";
  }
  const formPanel = document.getElementById("prescriptionForm");
  if (formPanel) formPanel.style.display = "block";

  await loadNotes();
  await loadPrescriptions();
  await loadQuestionnaires();

  try {
```

```
    if (detailsPanel) {
      detailsPanel.scrollIntoView({ behavior: "smooth", block: "start" });
    }
  } catch (_) {}
}

// Sprint 2, Steven An
//  Notes (with delete)
async function loadNotes() {
  if (!notesList || !selectedPatient) return;
  notesList.innerHTML = "";
  const token = getAccessToken();

  const rows = await supabaseRequest({
    method: "GET",
    path: "patient_doctor_assignments",
    accessToken: token,
    query:
`?select=id,notes&patient_userid=eq.${encodeURIComponent(selectedPatient.userid)}&doctor
_userid=eq.${encodeURIComponent(doctorUserId)}&limit=1`
  });

  if (!rows || rows.length === 0) return;

  const assignment = rows[0];
  const notes = assignment.notes || "";
```

```
  const lines = notes.split("\n").filter(l => l.trim() !== "");
  lines.forEach((line, idx) => {
    const li = document.createElement("li");

    const textSpan = document.createElement("span");
    textSpan.textContent = line;

    const delBtn = document.createElement("button");
    delBtn.className = "small-btn";
    delBtn.textContent = "Delete";
    delBtn.addEventListener("click", () => deleteNote(idx));

    li.appendChild(textSpan);
    li.appendChild(delBtn);
    notesList.appendChild(li);
  });
}

async function deleteNote(noteIndex) {
  if (!selectedPatient) return;
  const token = getAccessToken();

  const rows = await supabaseRequest({
    method: "GET",
```

```
  path: "patient_doctor_assignments",
  accessToken: token,
  query:
`?select=id,notes&patient_userid=eq.${encodeURIComponent(selectedPatient.userid)}&doctor
_userid=eq.${encodeURIComponent(doctorUserId)}&limit=1`
 });

 if (!rows || rows.length === 0) return;

 const assignmentId = rows[0].id;
 const current = rows[0].notes || "";
 const lines = current.split("\n").filter(l => l.trim() !== "");

 if (noteIndex < 0 || noteIndex >= lines.length) return;

 lines.splice(noteIndex, 1);
 const updated = lines.join("\n");

 await supabaseRequest({
  method: "PATCH",
  path: "patient_doctor_assignments",
  accessToken: token,
  body: { notes: updated },
  query: `?id=eq.${assignmentId}`
 });
```

```
  await loadNotes();
}

async function onAddNote() {
  const note = (newNote && newNote.value || "").trim();
  if (!note || !selectedPatient) return;

  const token = getAccessToken();
  const rows = await supabaseRequest({
    method: "GET",
    path: "patient_doctor_assignments",
    accessToken: token,
    query:
`?select=id,notes&patient_userid=eq.${encodeURIComponent(selectedPatient.userid)}&doctor
_userid=eq.${encodeURIComponent(doctorUserId)}&limit=1`
  });

  if (!rows || rows.length === 0) return;

  const assignmentId = rows[0].id;
  const current = rows[0].notes || "";
  const ts = new Date().toISOString();
  const updated = (current ? current + "\n" : "") + `${ts} — ${note}`;

  await supabaseRequest({
    method: "PATCH",
```

```
    path: "patient_doctor_assignments",
    accessToken: token,
    body: { notes: updated },
    query: `?id=eq.${assignmentId}`
  });

  if (newNote) newNote.value = "";
  await loadNotes();
}

// Sprint 2, Steven An
//  Prescriptions (free-text medication name)
async function loadPrescriptions() {
  if (!rxList || !selectedPatient) return;
  rxList.innerHTML = "";
  const token = getAccessToken();

  const rows = await supabaseRequest({
    method: "GET",
    path: "user_medications",
    accessToken: token,
    query:
`?select=id,medicationid,dosage_mg,form,notes,start_date,end_date&userid=eq.${encodeURIC
omponent(selectedPatient.userid)}&order=id.desc`
  });
```

```
(rows || []).forEach(r => {
  const li = document.createElement("li");

  const left = document.createElement("div");
  const nameDiv = document.createElement("div");
  nameDiv.style.fontWeight = "600";
  nameDiv.textContent = medicationsById[r.medicationid] || `Medication #${r.medicationid}`;
  left.appendChild(nameDiv);

  const metaLines = [
    r.dosage_mg ? `${r.dosage_mg} mg` : null,
    r.form || null,
    r.start_date ? `start ${r.start_date}` : null,
    r.end_date ? `end ${r.end_date}` : null
  ].filter(Boolean);

  metaLines.forEach(line => {
    const d = document.createElement("div");
    d.textContent = line;
    left.appendChild(d);
  });

  if (r.notes) {
    const notesDiv = document.createElement("div");
    notesDiv.style.opacity = "0.85";
```

```
      notesDiv.textContent = r.notes;
      left.appendChild(notesDiv);
    }

    const removeBtn = document.createElement("button");
    removeBtn.className = "small-btn danger-btn remove-btn";
    removeBtn.style.color = "#b00020";
    removeBtn.style.borderColor = "#b00020";
    removeBtn.textContent = "x";
    removeBtn.addEventListener("click", async () => {
      await supabaseRequest({
        method: "DELETE",
        path: "user_medications",
        accessToken: token,
        query: `?id=eq.${encodeURIComponent(r.id)}`
      });
      await loadPrescriptions();
    });

    li.appendChild(left);
    li.appendChild(removeBtn);
    rxList.appendChild(li);
  });
}
```

```
async function onSavePrescription() {
  if (!selectedPatient) return;

  const token = getAccessToken();

  const medName = (rxMedicationName && rxMedicationName.value || "").trim();
  if (!medName) {
    alert("Please enter a medication name.");
    return;
  }

  // Find or create medication row by name
  let medicationid = null;

  const existing = await supabaseRequest({
    method: "GET",
    path: "medications",
    accessToken: token,
    query: `?select=id,name&name=eq.${encodeURIComponent(medName)}&limit=1`
  });

  if (existing && existing.length > 0) {
    medicationid = existing[0].id;
  } else {
    const inserted = await supabaseRequest({
```

```
    method: "POST",
    path: "medications",
    accessToken: token,
    body: [{ name: medName }]
  });
  if (inserted && inserted[0] && inserted[0].id !== undefined) {
    medicationid = inserted[0].id;
  }
}

if (medicationid == null) {
  alert("Unable to save medication.");
  return;
}

// Update cache
medicationsById[medicationid] = medName;

const dosage_mg = rxDosage && rxDosage.value ? parseFloat(rxDosage.value) : null;
const form      = (rxForm && rxForm.value  || "").trim() || null;
const notes     = (rxNotes && rxNotes.value || "").trim() || null;
const start     = rxStart && rxStart.value || null;
const end       = rxEnd && rxEnd.value   || null;

await supabaseRequest({
```

```
    method: "POST",
    path: "user_medications",
    accessToken: token,
    body: [{
      userid:      selectedPatient.userid,
      medicationid: medicationid,
      dosage_mg:    dosage_mg,
      form:        form,
      notes:       notes,
      start_date:   start,
      end_date:     end
    }]
  });

  if (rxMedicationName) rxMedicationName.value = "";
  if (rxDosage) rxDosage.value = "";
  if (rxForm) rxForm.value = "";
  if (rxNotes) rxNotes.value = "";
  if (rxStart) rxStart.value = "";
  if (rxEnd) rxEnd.value = "";

  await loadPrescriptions();
}

// Sprint 3, Steven An, Eitan Pupkin
```

```
// --------- Questionnaire responses ----------
async function loadQuestionnaires() {
  if (!selectedPatient) return;
  const token = getAccessToken();

  const rows = await supabaseRequest({
    method: "GET",
    path: "questionnaire_responses",
    accessToken: token,
    query:
`?select=answers,created_at&patient_userid=eq.${encodeURIComponent(selectedPatient.userid)}&order=created_at.desc`
  });

  const qList = document.getElementById("questionnaireList");
  if (!qList) return;
  qList.innerHTML = "";

  (rows || []).forEach(r => {
    const a  = r.answers || {};
    const li = document.createElement("li");

    if (a.type === "medication_survey") {
      // Medication survey view
      li.innerHTML = `
        <div><strong>${new Date(r.created_at).toLocaleString()}</strong></div>
```

```
      <div>Medication: ${a.medication_name || ("#" + a.medicationid)}</div>
      <div>Adherence: ${a.adherence ?? "N/A"}</div>
      <div>Effectiveness: ${a.effectiveness ?? "N/A"}</div>
      <div>Side effects: ${a.side_effects || "None reported"}</div>
    `;
  } else {
    // Original health questionnaire
    li.innerHTML = `
      <div><strong>${new Date(r.created_at).toLocaleString()}</strong></div>
      <div>Allergies: ${a.allergies || "N/A"}</div>
      <div>Symptoms: ${a.symptoms || "N/A"}</div>
      <div>OTC meds: ${a.otc || "N/A"}</div>
    `;
  }

  qList.appendChild(li);
 });
}

// Sprint 2, Steven An
// Allows Doctor to check for drug interactions using openFDA API
// --------- FDA Drug Interaction Checker ----------
async function handleInteractionCheck() {
 const medsInput  = document.getElementById("interactionInput");
 const resultsDiv = document.getElementById("interactionResults");
```

```
const meds = medsInput.value.trim();
if (!meds) {
  resultsDiv.innerHTML = "<p>Please enter medications.</p>";
  return;
}

const list = meds.split(",").map(x => x.trim()).filter(Boolean);
resultsDiv.innerHTML = "<p>Checking...</p>";

const data = await checkFDAInteractions(meds);

// Single-drug behavior (now with fallback to warnings)
if (list.length === 1) {
  const medName = list[0];
  const info = data.results && data.results[medName];

  const interactions  = (info && info.interactions)   || [];
  const fallbackTexts = (info && info.fallbackTexts)  || [];

  // Prefer drug_interactions if present; otherwise use fallback warning text
  let labelTexts = interactions.length ? interactions : fallbackTexts;

  if (!labelTexts.length) {
    resultsDiv.innerHTML = `
```

```
      <p>No interaction or warning information was found in the FDA label fields we checked for this medication.</p>
    `;
    return;
  }

  let html = `<h4>FDA label information for ${medName}</h4>`;

  labelTexts.forEach(txt => {
    const shortText = truncate(txt, 150);
    const fullText  = txt.replace(/\n/g, "<br>");

    html += `
      <div class="interaction-item">
        <p>
          <span class="short-text">${shortText}</span>
          <span class="full-text" style="display:none;">${fullText}</span>
        </p>
        <button class="toggle-btn" onclick="toggleInteraction(this)">Show More</button>
      </div>
    `;
  });

  resultsDiv.innerHTML = html;
  return; // don't run multi-drug logic
 }
```

```
// Existing multi-drug behavior
if (!data.interactionPairs.length) {
  resultsDiv.innerHTML = "<p>No cross-interactions detected.</p>";
  return;
}

let html = "<h4>Detected Interactions</h4>";

data.interactionPairs.forEach(pair => {
  const shortText = truncate(pair.text, 150);
  const fullText  = pair.text.replace(/\n/g, "<br>");

  html += `
    <div class="interaction-item">
      <p>
        <strong>${pair.from} ↔ ${pair.to}:</strong>
        <span class="short-text">${shortText}</span>
        <span class="full-text" style="display:none;">${fullText}</span>
      </p>
      <button class="toggle-btn" onclick="toggleInteraction(this)">Show More</button>
    </div>
  `;
});
```

```
  resultsDiv.innerHTML = html;
}

// Sprint 2, Steven An
// FDA Drug Interaction Checker
async function checkFDAInteractions(meds) {
  const list = meds.split(",").map(x => x.trim()).filter(Boolean);
  const results = {};
  const pairs = [];

  for (const med of list) {
    const url =
`https://api.fda.gov/drug/label.json?search=openfda.generic_name:"${med.toUpperCase()}"&limit=3`;

    try {
      const res = await fetch(url);
      const data = await res.json();

      if (!data.results || !data.results[0]) {
        results[med] = { error: "No data found", interactions: [], fallbackTexts: [] };
        continue;
      }

      const info = data.results[0];
```

```
// Primary interaction field (what you were using before)
const interactions = info.drug_interactions || [];

// Fallback: pull extra warning-related fields if needed
const fallbackTexts = [];
function addField(fieldName) {
  const val = info[fieldName];
  if (!val) return;
  if (Array.isArray(val)) {
    val.forEach(v => {
      if (typeof v === "string") fallbackTexts.push(v);
    });
  } else if (typeof val === "string") {
    fallbackTexts.push(val);
  }
}

// These are common places where ibuprofen / NSAID warnings live
addField("warnings_and_cautions");
addField("warnings");
addField("boxed_warning");
addField("precautions");
addField("contraindications");

results[med] = {
```

```
      interactions: interactions,
      fallbackTexts: fallbackTexts
    };
  } catch (err) {
    results[med] = { error: err.message, interactions: [], fallbackTexts: [] };
  }
}

// Build interaction pairs (unchanged logic)
for (const m1 of list) {
  for (const m2 of list) {
    if (m1 === m2) continue;
    const texts = results[m1]?.interactions || [];

    texts.forEach(txt => {
      if (txt.toLowerCase().includes(m2.toLowerCase())) {
        pairs.push({ from: m1, to: m2, text: txt });
      }
    });
  }
}

return { results, interactionPairs: pairs };
}
```

home.js

```
/*
   home.js   Virginia Tech      Sprint 1,2,3
   Main landing page with login form and redirection based on role assignment

   Home page that allows user logins and redirects based on role- doctor, administrator, patient
*/

document.getElementById("loginForm").addEventListener("submit", async (e) => {
   e.preventDefault();
   const email = document.getElementById("email").value.trim();
   const password = document.getElementById("password").value;

   try {
      const signIn = await supabaseAuthSignIn(email, password);
      if (signIn.error) throw new Error(signIn.error_description || signIn.error || "Login failed");

      const session = signIn;
      if (!session || !session.access_token) throw new Error("No access token returned");
      saveSession(session, email);
      // Ensure profile exists before redirect
      const token = getAccessToken();
      let users = await supabaseRequest({
         method: "GET",
         path: "user_accounts",
```

```
        accessToken: token,
        query: `?select=id,userid,role,fullname,email&email=eq.${encodeURIComponent(email)}`
    });
    if (!users || users.length === 0) {
        try {
            const pending = localStorage.getItem('pendingProfile');
            let profile = pending ? JSON.parse(pending) : null;
            if (!profile) {
                profile = { userid: generateUserId(), email: email, fullname: email, phonenumber: null, role: 'patient' };
            }
            await supabaseRequest({ method: 'POST', path: 'user_accounts', accessToken: token, body: [profile] });
            users = [profile];
        } catch (_) {  }
    }

    // Redirect to appropriate home page based on role
    const role = (users && users[0] && users[0].role) || 'patient';
    try { localStorage.setItem('sbRole', role); } catch (_) {}
    if (role === 'doctor') {
        window.location.href = "doctor.html";
    }else if (role === 'administrator') {
        window.location.href = "admin.html";
    }  else {
```

```
            window.location.href = "patient.html";
        }
    } catch (err) {
        alert(`Login error: ${err.message}`);
    }
});

// Redirect to appropritae role page if curr logged in using access token
(async function redirectIfLoggedIn() {
    try {
        const token = getAccessToken?.();
        const email = getEmail?.();
        if (!token || !email) return;

        const users = await supabaseRequest({
            method: "GET",
            path: "user_accounts",
            accessToken: token,
            query: `?select=role,email&email=eq.${encodeURIComponent(email)}&limit=1`
        });

        const role = (users && users[0] && users[0].role) || "patient";
        try { localStorage.setItem('sbRole', role); } catch (_) {}
        window.location.href =
            role === "doctor" ? "doctor.html" :
```

```
            role === "administrator" ? "admin.html" :
            "patient.html";
    } catch (_) {
    }
})();
```

logo.js

```
//Everett Miceli
// Top left SVG logo -- when clicked, returns to home page


const logoSVG = "logo2.svg";


(function () {
  function ensureLogo() {
    //if (currentPage === "home.html") return; // don't display on home page itself
    if (document.getElementById("appLogo")) return;
    const img = document.createElement("img");
    img.id = "appLogo";
    img.src = logoSVG;
    img.className = "app-logo";
    img.addEventListener("click", onLogoClick);
    document.body.appendChild(img);
  }


  function onLogoClick() { // Formerly redircted to home page, but function serves to redirect directly to role page for efficiency
    try {
      var role = localStorage.getItem('sbRole');
      var token = (typeof getAccessToken === 'function') ? getAccessToken() : localStorage.getItem('sbAccessToken');
      if (role && token) {
        var r = role.toLowerCase();
```

```
  if (r === 'doctor') { window.location.href = 'doctor.html'; return; }
  if (r === 'administrator') { window.location.href = 'admin.html'; return; }
  window.location.href = 'patient.html'; return;
 }
 // If no cached role but session exists, resolve role directly
 if (typeof supabaseRequest === 'function' && typeof getEmail === 'function') {
  var email = getEmail();
  if (token && email) {
   supabaseRequest({
    method: 'GET',
    path: 'user_accounts',
    accessToken: token,
    query: `?select=role,email&email=eq.${encodeURIComponent(email)}&limit=1`
   }).then(function(rows){
    var rr = (rows && rows[0] && rows[0].role) || 'patient';
    try { localStorage.setItem('sbRole', rr); } catch (_) {}
    rr = rr.toLowerCase();
    if (rr === 'doctor') { window.location.href = 'doctor.html'; return; }
    if (rr === 'administrator') { window.location.href = 'admin.html'; return; }
    window.location.href = 'patient.html';
   }).catch(function(){ window.location.href = 'home.html'; });
   return;
  }
 }
} catch (_) {}
```

```
    window.location.href = "home.html";
  }

  if (document.readyState === "loading") {
    document.addEventListener("DOMContentLoaded", ensureLogo);
  } else {
    ensureLogo();
  }
})();
```

patient.js

```
/*
  patient.js     Virginia Tech      Sprint 1,2,3
  Patient portal, this file handles all patient-related frontend logic and UI

  Allows patients to view their assigned doctor, see their prescriptions, and submit questionnaires.
  Also includes medication safety warning logic based on FDA data.
*/

const logoutBtn  = document.getElementById('logoutBtn');
const headerRole = document.getElementById('headerRole');
const doctorInfo = document.getElementById('doctorInfo');
const rxList     = document.getElementById('rxList');
// Medication survey state + elements Eitan Pupkin sprint 3
let currentMedicationForSurvey = null;   // { medicationId, medicationName, userid }
let currentMedicationSurveyId  = null;   // existing questionnaire_responses.id for this month (if any)

const medSurveyPanel          = document.getElementById("medSurveyPanel");
const medSurveyMedicationLabel = document.getElementById("medSurveyMedicationLabel");
const surveyAdherence         = document.getElementById("surveyAdherence");
const surveyEffectiveness     = document.getElementById("surveyEffectiveness");
const surveySideEffects       = document.getElementById("surveySideEffects");
const surveySaveBtn           = document.getElementById("surveySaveBtn");
init();
```

```
async function init() {
  const token = getAccessToken();
  const email = getEmail();
  if (!token || !email) {
    window.location.href = 'home.html';
    return;
  }

  logoutBtn.addEventListener('click', () => {
    clearSession();
    window.location.href = 'home.html';
  });

  // Resolve current user
  const users = await supabaseRequest({
    method: 'GET',
    path: 'user_accounts',
    accessToken: token,
    query: `?select=userid,fullname,role&email=eq.${encodeURIComponent(email)}`
  });
  if (!users || users.length === 0) {
    alert('No profile');
    return;
  }
```

```
  const me = users[0];
  headerRole.textContent = `Patient: ${me.fullname || email}`;

  // Sprint 3, Steven An
  // Medication Safety Warnings based on patient's prescriptions
  await loadPatientInteractionWarnings(me.userid);

  // Doctor assignment
  const assigns = await supabaseRequest({
    method: 'GET',
    path: 'patient_doctor_assignments',
    accessToken: token,
    query:
`?select=doctor_userid,patient_userid&patient_userid=eq.${encodeURIComponent(me.userid)}
`
  });
  if (assigns && assigns.length > 0) {
    const docId = assigns[0].doctor_userid;
    const docs = await supabaseRequest({
      method: 'GET',
      path: 'user_accounts',
      accessToken: token,
      query: `?select=fullname,email&userid=eq.${encodeURIComponent(docId)}`
    });
    if (docs && docs[0]) {
      doctorInfo.textContent = `Doctor: ${docs[0].fullname || docs[0].email}`;
```

```
    } else {
      doctorInfo.textContent = 'Doctor: Unknown';
    }
  } else {
    doctorInfo.textContent = 'Doctor: Not assigned';
  }

  // Prescriptions list
  const meds = await supabaseRequest({
    method: 'GET',
    path: 'medications',
    accessToken: token,
    query: '?select=id,name'
  });
  const medsById = {};
  (meds || []).forEach(m => { medsById[m.id] = m.name; });

  const rx = await supabaseRequest({
    method: 'GET',
    path: 'user_medications',
    accessToken: token,
    query:
`?select=medicationid,dosage_mg,form,notes,start_date,end_date&userid=eq.${encodeURICo
mponent(me.userid)}&order=id.desc`
  });
```

```
//updated by Eitan Pupkin sprint 3
rxList.innerHTML = '';
(rx || []).forEach(r => {
  const li = document.createElement('li');
  const name = medsById[r.medicationid] || `#${r.medicationid}`;

  const title = document.createElement('div');
  title.style.fontWeight = '600';
  title.textContent = name;

  // Multi-line meta
  const parts = [
    r.dosage_mg ? `${r.dosage_mg}mg` : null,
    r.form,
    r.start_date ? `start ${r.start_date}` : null,
    r.end_date ? `end ${r.end_date}` : null
  ].filter(Boolean);

  const notes = document.createElement('div');
  notes.style.opacity = '0.85';
  notes.textContent = r.notes ? r.notes : '';

  li.appendChild(title);
  parts.forEach(line => {
    const d = document.createElement('div');
```

```
    d.textContent = line;
    li.appendChild(d);
  });
  li.appendChild(notes);

  // Survey button for this medication
  const surveyBtn = document.createElement('button');
  surveyBtn.className = 'small-btn';
  surveyBtn.type = 'button';
  surveyBtn.textContent = 'Survey';
  surveyBtn.addEventListener('click', () => {
    openMedicationSurvey({
      medicationId: r.medicationid,
      medicationName: name,
      userid: me.userid
    });
  });
  li.appendChild(surveyBtn);

  rxList.appendChild(li);
});

// Sprint 3, Steven An
// Questionnaire submission logic
const questionnaireForm = document.getElementById("questionnaireForm");
```

```
if (questionnaireForm) {
  questionnaireForm.addEventListener("submit", async (e) => {
    e.preventDefault();

    const token2 = getAccessToken();
    const answers = {
      allergies: document.getElementById("q_allergies").value.trim(),
      symptoms: document.getElementById("q_symptoms").value.trim(),
      otc: document.getElementById("q_otc").value.trim()
    };

    await supabaseRequest({
      method: "POST",
      path: "questionnaire_responses",
      accessToken: token2,
      body: [{
        patient_userid: me.userid,
        answers: answers
      }]
    });

    alert("Your responses have been submitted.");
    questionnaireForm.reset();
  });
}
```

```
  if (surveySaveBtn) {
    console.log("Wiring surveySaveBtn click");
    surveySaveBtn.addEventListener("click", saveMedicationSurvey);
  }
}

// Sprint 3,Steven An
// Summarize the long FDA label text into a short patient-friendly warning
// Medication Safety Warnings based on FDA data
function summarizeInteractionText(raw, medName) {
  if (!raw) return null;

  const textLower    = raw.toLowerCase();
  const medNameLower = (medName || "").toLowerCase();

  // Opioid-specific safety
  const OPIOID_KEYWORDS = [
    "hydrocodone", "oxycodone", "morphine", "codeine",
    "fentanyl", "hydromorphone", "oxymorphone",
    "tramadol", "methadone", "buprenorphine"
  ];

  const isOpioid =
    OPIOID_KEYWORDS.some(o => medNameLower.includes(o)) ||
    textLower.includes("opioid");
```

```
  if (isOpioid) {
    return {
      title: "Opioid safety warning",
      text: "This medication is an opioid. Opioids can cause dangerous drowsiness and slowed breathing. Avoid alcohol, benzodiazepines, and other medicines that make you sleepy unless your doctor specifically tells you it is safe."
    };
  }

  // High-priority interaction concepts from FDA text

  if (textLower.includes("alcohol") || textLower.includes("ethanol")) {
    return {
      title: "Alcohol and your medication",
      text: "Do not drink alcohol while taking this medication. The combination can cause dangerous drowsiness, breathing problems, or overdose."
    };
  }

  if (textLower.includes("benzodiazepine") ||
      textLower.includes("diazepam") ||
      textLower.includes("lorazepam") ||
      textLower.includes("alprazolam") ||
      textLower.includes("clonazepam")) {
    return {
```

```
    title: "Anxiety / sleep medicines",
    text: "This medication may interact with benzodiazepines (anxiety or sleep medicines). Together they can slow your breathing and be life-threatening. Do not combine them without talking to your doctor."
  };
 }

 if (textLower.includes("cns depressant")) {
  return {
    title: "Other medicines that make you sleepy",
    text: "This medicine may interact with other drugs that make you sleepy (like strong pain medicines, sleeping pills, or some anxiety medicines). Using them together can be dangerous without medical advice."
  };
 }

 if (textLower.includes("bleeding") ||
    textLower.includes("anticoagulant") ||
    textLower.includes("warfarin")) {
  return {
    title: "Bleeding risk",
    text: "This combination can increase your risk of serious bleeding. Watch for unusual bruising or bleeding and contact your doctor or pharmacist."
  };
 }

 if (textLower.includes("serotonin syndrome")) {
```

```
    return {
      title: "Serotonin syndrome risk",
      text: "This combination can raise serotonin levels too much and cause a serious condition called serotonin syndrome. If you feel very agitated, confused, or have a fast heartbeat or fever, contact a doctor immediately."
    };
  }

  // Fallback: compress the first sentence of whatever FDA said
  const firstSentence = raw.split(/[\.\n]/)[0];
  const cleaned = firstSentence.replace(/\s+/g, " ").trim();
  if (!cleaned) return null;

  if (cleaned.length <= 200) {
    return { title: "Interaction notice", text: cleaned };
  }

  return {
    title: "Interaction notice",
    text: cleaned.slice(0, 197) + "..."
  };
}

// Sprint 3, Steven An
// Call openFDA for the patient's meds and collect relevant label text per med
async function checkFDAInteractionsForPatient(meds) {
```

```
  const list = meds.split(",").map(x => x.trim()).filter(Boolean);
  const results = {};

  for (const med of list) {
    const url =
`https://api.fda.gov/drug/label.json?search=openfda.generic_name:"${med.toUpperCase()}"&limit=3`;

    try {
      const res = await fetch(url);
      const data = await res.json();

      if (!data.results || !data.results[0]) {
        results[med] = { error: "No data found", texts: [] };
        continue;
      }

      const info = data.results[0];

      // Collect text from multiple fields (Option C)
      const textPieces = [];

      function addField(fieldName) {
        const val = info[fieldName];
        if (!val) return;
        if (Array.isArray(val)) {
```

```
    val.forEach(v => {
      if (typeof v === "string") textPieces.push(v);
    });
  } else if (typeof val === "string") {
    textPieces.push(val);
  }
}

addField("drug_interactions");
addField("warnings_and_cautions");
addField("warnings");
addField("boxed_warning");
addField("precautions");
addField("clinical_pharmacology");
addField("contraindications");

// Deduplicate text chunks
const seen = new Set();
const uniqueTexts = [];
textPieces.forEach(t => {
  const key = t.slice(0, 200); // crude but enough
  if (seen.has(key)) return;
  seen.add(key);
  uniqueTexts.push(t);
});
```

```
      results[med] = {
        texts: uniqueTexts
      };
    } catch (err) {
      console.error("openFDA error for", med, err);
      results[med] = { error: err.message, texts: [] };
    }
  }

  return { results };
}

// Sprint 3, Steven An
// Main entry point for patient safety warnings based on FDA data
async function loadPatientInteractionWarnings(patientUserId) {
  const token = getAccessToken();
  const safetyDiv = document.getElementById("safetyWarnings");
  if (!safetyDiv) return;

  safetyDiv.textContent = "Checking your medications for important safety warnings...";

  // All medications (id -> name)
  const medRows = await supabaseRequest({
    method: "GET",
```

```
  path: "medications",
  accessToken: token,
  query: "?select=id,name"
});

const medicationsById = {};
(medRows || []).forEach(m => {
  medicationsById[m.id] = m.name;
});

// This patient's prescriptions
const userMeds = await supabaseRequest({
  method: "GET",
  path: "user_medications",
  accessToken: token,
  query: `?select=id,medicationid&userid=eq.${encodeURIComponent(patientUserId)}`
});

const medNames = (userMeds || [])
  .map(r => medicationsById[r.medicationid])
  .filter(Boolean);

if (medNames.length === 0) {
  safetyDiv.textContent = "You currently have no medications on file.";
  return;
```

```
  }

  // Call openFDA once we know all med names
  let data;
  try {
    data = await checkFDAInteractionsForPatient(medNames.join(", "));
  } catch (err) {
    console.error("Interaction check failed:", err);
    safetyDiv.innerHTML = `
      <p>We couldn't check interaction warnings right now.</p>
      <p class="tiny-muted">Please try again later or ask your doctor or pharmacist if you have questions.</p>
    `;
    return;
  }

  const seen = new Set();
  const warnings = [];

  // For each medication, for each relevant text block, summarize it
  Object.entries(data.results || {}).forEach(([medName, info]) => {
    (info.texts || []).forEach(txt => {
      const summary = summarizeInteractionText(txt, medName);
      if (!summary) return;

      const key = summary.title + "::" + summary.text;
```

```
    if (seen.has(key)) return;
    seen.add(key);

    warnings.push(summary);
  });
});

// Render warnings
if (warnings.length === 0) {
  safetyDiv.innerHTML = `
    <p>No special interaction warnings were flagged based on your current medications.</p>
    <p class="tiny-muted">
      This does not replace advice from your doctor or pharmacist.
    </p>
  `;
} else {
  safetyDiv.innerHTML = `
    ${warnings.map(w => `
      <div class="warning-card">
        <strong>⚠ ${w.title}</strong>
        <p>${w.text}</p>
      </div>
    `).join("")}
    <p class="tiny-muted">
      These warnings are for information only and do not replace advice from your doctor or pharmacist.
```

```
      Do not start, stop, or change any medication without talking to them.
    </p>
   `;
  }
}
//helper functions for surveys Eitan Pupkin
async function openMedicationSurvey(info) {
  currentMedicationForSurvey = info;
  currentMedicationSurveyId = null; // reset

  if (medSurveyPanel) {
    medSurveyPanel.style.display = 'block';
  }
  if (medSurveyMedicationLabel) {
    medSurveyMedicationLabel.textContent = `Medication: ${info.medicationName}`;
  }

  // Clear fields
  if (surveyAdherence)     surveyAdherence.value = '';
  if (surveyEffectiveness) surveyEffectiveness.value = '';
  if (surveySideEffects)   surveySideEffects.value = '';

  // Load this-month survey (if exists)
  const token = getAccessToken();
  const rows = await supabaseRequest({
```

```
  method: "GET",
  path: "questionnaire_responses",
  accessToken: token,
  query:
`?select=id,answers,created_at&patient_userid=eq.${encodeURIComponent(info.userid)}&orde
r=created_at.desc`
 });

 const now = new Date();
 const thisYear  = now.getFullYear();
 const thisMonth = now.getMonth(); // 0-11

 let latestThisMonth = null;

 (rows || []).forEach(r => {
  const a = r.answers || {};
  if (a.type !== "medication_survey") return;
  if (a.medicationid !== info.medicationId) return;

  const d = new Date(r.created_at);
  if (d.getFullYear() === thisYear && d.getMonth() === thisMonth) {
   if (!latestThisMonth || d > new Date(latestThisMonth.created_at)) {
    latestThisMonth = r;
   }
  }
 });
```

```
  if (latestThisMonth) {
    currentMedicationSurveyId = latestThisMonth.id;
    const a = latestThisMonth.answers || {};
    if (surveyAdherence)     surveyAdherence.value     = a.adherence ?? '';
    if (surveyEffectiveness) surveyEffectiveness.value = a.effectiveness ?? '';
    if (surveySideEffects)   surveySideEffects.value   = a.side_effects ?? '';
  }
}

async function saveMedicationSurvey() {
  console.log("saveMedicationSurvey() called");
  if (!currentMedicationForSurvey) return;

  const token = getAccessToken();

  const adherence = surveyAdherence && surveyAdherence.value
    ? parseInt(surveyAdherence.value, 10)
    : null;
  const effectiveness = surveyEffectiveness && surveyEffectiveness.value
    ? parseInt(surveyEffectiveness.value, 10)
    : null;
  const sideEffects = surveySideEffects && surveySideEffects.value
    ? surveySideEffects.value.trim()
    : null;
```

```
const answers = {
  type: "medication_survey",
  medicationid:   currentMedicationForSurvey.medicationId,
  medication_name: currentMedicationForSurvey.medicationName,
  adherence,
  effectiveness,
  side_effects: sideEffects
};

if (currentMedicationSurveyId) {
  // Update existing this-month survey
  await supabaseRequest({
    method: "PATCH",
    path: "questionnaire_responses",
    accessToken: token,
    query: `?id=eq.${encodeURIComponent(currentMedicationSurveyId)}`,
    body: {
      answers: answers,
      created_at: new Date().toISOString()
    }
  });
} else {
  // Create new survey for this month
  await supabaseRequest({
```

```
      method: "POST",
      path: "questionnaire_responses",
      accessToken: token,
      body: [{
        patient_userid: currentMedicationForSurvey.userid,
        answers: answers
      }]
    });
  }

  alert("Medication survey saved for this month.");
}
```

register.js

```
//  Everett Miceli & Rahul Palani

// Supabase utilized for registration. Email confirmation can be disabled via Supabase

// Supabase user/registration handled within "Authentication" table. Seperate table needed for rest of operations (user_accounts) in Table Editor


document.getElementById("registerForm").addEventListener("submit", async (e) => {
  e.preventDefault();


  const fullName = document.getElementById("name").value.trim();
  const email = document.getElementById("email").value.trim();
  const userId = document.getElementById("userId").value.trim();
  const phoneNumber = (document.getElementById("phoneNumber")?.value || "").trim();
  const password = document.getElementById("password").value;
  const role = document.getElementById("role").value;


  if (!userId) {
    alert("Please enter a unique User ID.");
    return;
  }


  try {
    //cache profile data in case email confirmation is enabled
    const pendingProfile = {
      userid: userId,
      email: email,
```

```
    fullname: fullName,
    phonenumber: phoneNumber || null,
    role: role
};
localStorage.setItem("pendingProfile", JSON.stringify(pendingProfile));

//preemptively check for duplicate userids
try {
    const existingUserId = await supabaseRequest({
        method: "GET",
        path: "user_accounts",
        query: `?select=id&userid=eq.${encodeURIComponent(userId)}`
    });
    if (existingUserId && existingUserId.length > 0) {
        alert("That username is taken. Please choose another.");
        return;
    }
} catch (_) {}

//premptively check for duplicate emails
try {
    const existingProfile = await supabaseRequest({
        method: "GET",
        path: "user_accounts",
        query: `?select=id&email=eq.${encodeURIComponent(email)}`
```

```
      });
      if (existingProfile && existingProfile.length > 0) {
        alert("An account with this email already exists. Please log in instead.");
        window.location.href = "home.html";
        return;
      }
    } catch (_) {}

    //sign up with Supabase Auth
    const origin = window.location.origin || (window.location.protocol + '//' +
window.location.host);
    const redirectTo = origin + '/auth-callback.html';
    const signUp = await supabaseAuthSignUp(email, password, redirectTo);
    if (signUp.error) throw new Error(signUp.error.message || "Signup failed");

    //use session access token in the event email confirms are disabled
    const session = signUp.session || signUp;
    if (!session || !session.access_token) {
      alert("Account created. Please check your email to confirm before logging in.");
      window.location.href = "home.html";
      return;
    }

    saveSession(session, email);

    //insert profile row into user_accounts table
```

```
const accessToken = getAccessToken();
const profile = [{
   userid: userId,
   email: email,
   fullname: fullName,
   phonenumber: phoneNumber || null,
   role: role
}];

//do not insert if email is already in use
const existing = await supabaseRequest({
   method: "GET",
   path: "user_accounts",
   accessToken,
   query: `?select=id&email=eq.${encodeURIComponent(email)}`
});
if (!existing || existing.length === 0) {
   //do not insert if userid is already in use
   const existingByUserId = await supabaseRequest({
      method: "GET",
      path: "user_accounts",
      accessToken,
      query: `?select=id&userid=eq.${encodeURIComponent(userId)}`
   });
   if (existingByUserId && existingByUserId.length > 0) {
```

```
            alert("That username is taken. Please choose another.");
            return;
          }
          await supabaseRequest({
            method: "POST",
            path: "user_accounts",
            body: profile,
            accessToken
          });
        } else {
          alert("An account with this email already exists. Please log in instead.");
          window.location.href = "home.html";
          return;
        }

        localStorage.removeItem("pendingProfile");

        alert("Account created successfully.");
        window.location.href = "home.html";
      } catch (err) {
        alert(`Registration error: ${err.message}`);
      }
    });
```

reminders.js

```
// Everett Miceli Virginia Tech Sprint 3
// Very basic reminder functionality, does not currently remind
// Saves reminders/configs to patient_reminders table and displays them
// TODO: future functionality/integration with whatever its deployed within (mobile app, ect)
(function () {
  var headerRole, logoutBtn, setReminderBtn, userId, email, token, reminderList;
  document.addEventListener('DOMContentLoaded', async function () {
    headerRole = document.getElementById('headerRole');
    logoutBtn = document.getElementById('logoutBtn');
    setReminderBtn = document.getElementById('setReminderBtn');
    reminderList = document.getElementById('reminderList');
    token = typeof getAccessToken === 'function' ? getAccessToken() : null;
    email = typeof getEmail === 'function' ? getEmail() : null;
    if (!token || !email) { window.location.href = 'home.html'; return; }
    if (logoutBtn) {
      logoutBtn.addEventListener('click', function () {
        try { if (typeof clearSession === 'function') clearSession(); } catch (_) {}
        window.location.href = 'home.html';
      });
    }
    try {
      var rows = await supabaseRequest({
        method: 'GET',
        path: 'user_accounts',
```

```
      accessToken: token,
      query: `?select=userid,fullname&email=eq.${encodeURIComponent(email)}&limit=1`
    });
    if (rows && rows[0]) {
      userId = rows[0].userid;
      if (headerRole) headerRole.textContent = 'Patient: ' + (rows[0].fullname || email);
    } else {
      if (headerRole) headerRole.textContent = 'Patient: ' + email;
    }
  } catch (_) {
    if (headerRole) headerRole.textContent = 'Patient: ' + email;
  }
  if (setReminderBtn) setReminderBtn.addEventListener('click', onSave);
  await loadReminders();
});
async function onSave() {
  if (!userId) return;
  var medication = (document.getElementById('rem_medication') || {}).value || '';
  var frequency = (document.getElementById('rem_frequency') || {}).value || 'once_daily';
  var times = (document.getElementById('rem_times') || {}).value || '';
  var withMeal = ((document.getElementById('rem_with_meal') || {}).value || 'false') ===
'true';
  var notes = (document.getElementById('rem_notes') || {}).value || '';
  medication = medication.trim();
  times = times.trim();
  notes = notes.trim();
```

```
  if (!medication || !times) { alert('Enter medication and times.'); return; }
  try {
    await supabaseRequest({
      method: 'POST',
      path: 'patient_reminders',
      accessToken: token,
      body: [{
        patient_userid: userId,
        medication: medication,
        frequency: frequency,
        times: times,
        with_meal: withMeal,
        notes: notes
      }]
    });
    await loadReminders();
  } catch (_) {
    alert('Save failed.');
  }
}
async function loadReminders() {
  if (!userId || !reminderList) return;
  try {
    var rows = await supabaseRequest({
      method: 'GET',
```

```
      path: 'patient_reminders',
      accessToken: token,
      query:
`?select=id,medication,frequency,times,with_meal,notes&patient_userid=eq.${encodeURIComponent(userId)}&order=id.desc`
    });
    render(rows || []);
  } catch (_) {
    render([]);
  }
}
function render(rows) {
  reminderList.innerHTML = '';
  if (!rows.length) {
    var li = document.createElement('li');
    li.textContent = 'No reminders yet.';
    reminderList.appendChild(li);
    return;
  }
  rows.forEach(function (r) {
    var li = document.createElement('li');
    var left = document.createElement('div');
    left.style.display = 'flex';
    left.style.flexDirection = 'column';
    left.style.alignItems = 'flex-start';
    left.innerHTML = '<strong>' + (r.medication || '') + '</strong>' +
```

```
      (r.times ? ('<div>Times: ' + r.times + '</div>') : '') +
      (r.frequency ? ('<div>Frequency: ' + r.frequency + '</div>') : '') +
      (r.with_meal ? '<div>With meal</div>' : '') +
      (r.notes ? ('<div>' + r.notes + '</div>') : '');
    var del = document.createElement('button');
    del.className = 'small-btn danger-btn';
    del.textContent = 'Remove';
    del.addEventListener('click', function () { removeReminder(r.id); });
    li.appendChild(left);
    li.appendChild(del);
    reminderList.appendChild(li);
  });
}
async function removeReminder(id) {
  if (!id) return;
  try {
    await supabaseRequest({
      method: 'DELETE',
      path: 'patient_reminders',
      accessToken: token,
      query: `?id=eq.${encodeURIComponent(id)}`
    });
    await loadReminders();
  } catch (_) {
    alert('Delete failed.');
```

```
    }
  }
})();
```

supabase.js

```
//  Everett Miceli

// Supabase authentication, token, and accessing functions

// Authentication settings (RLS, email confirmation, ect) handled within Supabase

// Anon key used for client side operations, use service key for future addition of back-end operations


const supabaseUrl = "https://zsghiwxyahwsogcmouny.supabase.co";

const supabaseAnonKey = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6InpzZ2hpd3h5YWh3c29nY21vdW55Iiwicm9sZSI6ImFub24iLCJpYXQiOjE3NTk4NTMxNjksImV4cCI6MjA3NTQyOTE2OX0.3BpLsHGkPDV2cLCWhtUpGaKOJB-1Y0mxRaD5yO4P-Cs";


function supabaseAuthSignUp(email, password, redirectTo) {
   return fetch(`${supabaseUrl}/auth/v1/signup`, {
      method: "POST",
      headers: {
         "Content-Type": "application/json",
         "apikey": supabaseAnonKey,
      },
      body: JSON.stringify({ email, password, options: redirectTo ? { emailRedirectTo: redirectTo } : undefined })
   }).then(r => r.json());
}


function supabaseAuthSignIn(email, password) {
   return fetch(`${supabaseUrl}/auth/v1/token?grant_type=password`, {
```

```
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "apikey": supabaseAnonKey,
      },
      body: JSON.stringify({ email, password })
    }).then(r => r.json());
}

function supabaseRequest({method = "GET", path, body = undefined, accessToken = undefined, query = ""}) {
    const headers = {
      "apikey": supabaseAnonKey,
      "Accept": "application/json",
    };
    if (accessToken) headers["Authorization"] = `Bearer ${accessToken}`;
    if (body !== undefined) {
      headers["Content-Type"] = "application/json";
      // return immediately after insert, fix to first click error when adding medication
      headers["Prefer"] = "return=representation";
    }

    const url = `${supabaseUrl}/rest/v1/${path}${query}`;
    return fetch(url, {
      method,
      headers,
```

```
    body: body !== undefined ? JSON.stringify(body) : undefined,
  }).then(async (r) => {
    const text = await r.text();
    let data = null;
    try { data = text ? JSON.parse(text) : null; } catch {}
    if (!r.ok) {
      const msg = (data && (data.message || data.error_description || data.error)) ||
r.statusText;
      throw new Error(msg);
    }
    return data;
  });
}

function saveSession(session, email) {
  if (!session) return;
  if (session.access_token) localStorage.setItem("sbAccessToken", session.access_token);
  if (session.refresh_token) localStorage.setItem("sbRefreshToken", session.refresh_token);
  if (email) localStorage.setItem("sbEmail", email);
}

function getAccessToken() {
  return localStorage.getItem("sbAccessToken");
}

function getEmail() {
```

```
    return localStorage.getItem("sbEmail");
}

function clearSession() {
    localStorage.removeItem("sbAccessToken");
    localStorage.removeItem("sbRefreshToken");
    localStorage.removeItem("sbEmail");
    try { localStorage.removeItem("sbRole"); } catch (_) {}
}

// Lightweight JWT (JSON web token) decoder (no validation, just base64 decode)
// Extract user info from token, not used for auth
function decodeJwt(token) {
    try {
        const parts = token.split('.');
        if (parts.length !== 3) return null;
        const payload = parts[1].replace(/-/g, '+').replace(/_/g, '/');
        const decoded = atob(payload);
        return JSON.parse(decoded);
    } catch (_) { return null; }
}

function generateUserId() {
    if (window.crypto && crypto.randomUUID) return crypto.randomUUID();
    return 'u_' + Math.random().toString(36).slice(2, 10);
```

```
}
```

admin.html

```
<!--
  admin.html   Virginia Tech       Sprint 1,2,3
  Admin portal for managing user accounts, including role assignment and deletion.

  Admin interface for managing user accounts, including searching, viewing details,
  changing roles, and deleting accounts.
  Everett Miceli
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Admin</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
<div class="topbar">
  <div class="role" id="headerRole"></div>
  <button id="logoutBtn" class="logout-btn">Logout</button>
  <button onclick="window.location.href='patient.html'">Patient</button>
  <button onclick="window.location.href='doctor.html'">Doctor</button>
</div>
```

```
<div class="page">
  <h2>Admin Portal</h2>
  <div class="grid" id="overviewSection">
    <div class="panel">
      <h3>All Accounts</h3>
      <div class="row"><input type="text" id="patientSearch" placeholder="Search by name, email or userid"></div>
      <ul id="myPatientsList" class="list"></ul>
    </div>
  </div>

  <div class="panel" id="detailsPanel" style="display:none;">
    <h3 id="detailTitle">Account Details</h3>
    <div id="patientDetails" class="muted">Select an account to view details.</div>
    <div class="row">
      <label>Role</label>
      <select id="adminRoleSelect">
        <option value="patient">patient</option>
        <option value="doctor">doctor</option>
        <option value="administrator">administrator</option>
      </select>
      <button id="saveRoleBtn">Save Role</button>
    </div>
    <div class="row">
      <button id="deleteAccountBtn" class="secondary">Delete Account</button>
    </div>
```

```
  </div>
</div>

<script src="js/supabase.js"></script>
<script src="js/logo.js"></script>
<script src="js/admin.js"></script>
</body>
</html>
```

doctor.html

```
<!--
  doctor.html     Virginia Tech      Sprint 1,2,3
  Doctor portal, this file handles all doctor-related web UI structure.

  Allows doctors to view unassigned patients, manage their assigned patients,
  add notes and prescriptions, and view patient questionnaire responses.
  Steven An & Everett Miceli
 -->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Doctor Portal</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
<div class="topbar">
  <div class="role" id="headerRole"></div>
  <button id="logoutBtn" class="logout-btn">Logout</button>
</div>

<div class="page">
  <h2>Doctor Portal</h2>
```

```
<!-- Overview: Unassigned + My Patients -->
<div class="grid" id="overviewSection">
  <div class="panel">
    <h3>Unassigned Patients</h3>
    <ul id="unassignedList" class="list"></ul>
  </div>

  <div class="panel">
    <h3>My Patients</h3>
    <div class="row">
      <input type="text" id="patientSearch" placeholder="Search by name or email">
    </div>
    <ul id="myPatientsList" class="list"></ul>
  </div>
</div>

<!-- Patient Details -->
<div class="panel" id="detailsPanel" style="display:none;">
  <h3 id="detailTitle">Patient Details</h3>
  <div id="patientDetails" class="muted">
    Select a patient to view notes and medications.
  </div>

  <!-- Current Medications -->
```

```
<div class="row">
  <h3>Current Medications</h3>
  <ul id="rxList" class="list"></ul>
</div>

<!-- Add Medication -->
<div class="row">
  <div class="panel" id="prescriptionForm" style="display:none;">
    <h3>Add Medication</h3>

    <div class="row">
      <label>Medication</label>
      <!-- Sprint 3, Steven An -->
      <input type="text" id="rxMedicationName" placeholder="Enter medication name">
    </div>

    <div class="row">
      <label>Dosage (mg)</label>
      <input type="number" id="rxDosage" min="0" step="1">
    </div>

    <div class="row">
      <label>Form</label>
      <input type="text" id="rxForm" placeholder="tablet, capsule, etc.">
    </div>
```

```
    <div class="row">
      <label>Notes</label>
      <input type="text" id="rxNotes" placeholder="instructions">
    </div>

    <div class="row">
      <label>Start</label>
      <input type="date" id="rxStart">
    </div>

    <div class="row">
      <label>End</label>
      <input type="date" id="rxEnd">
    </div>

    <div class="row">
      <button id="rxSave">Prescribe</button>
    </div>
  </div>
</div>

<!-- Notes  -->
<div class="row">
  <h3>Notes</h3>
```

```
  <ul id="notesList" class="list"></ul>
  <div class="row">
    <input type="text" id="newNote" placeholder="Add a note">
    <button id="addNoteBtn">Add</button>
  </div>
 </div>

 <!-- Sprint 3, Steven An -->
 <!-- Questionnaire Responses -->
 <div class="row">
  <h3>Questionnaire Responses</h3>
  <ul id="questionnaireList" class="list"></ul>
 </div>

 <!-- Sprint 2, Steven An -->
 <!-- FDA Drug Interaction Checker -->
 <div class="row">
  <h3>Drug Interaction Checker</h3>
  <input type="text" id="interactionInput"
       placeholder="Enter medications, e.g., ibuprofen, warfarin">
  <button id="interactionBtn">Check Interactions</button>

  <div id="interactionResults" class="panel" style="margin-top:10px;"></div>
 </div>
</div>
```

```
</div>


<script src="js/supabase.js"></script>

<script src="js/logo.js"></script>

<script src="js/doctor.js"></script>

</body>

</html>
```

home.html

```
<!--
  home.html   Virginia Tech       Sprint 1,2,3
  Main landing page with login form and redirection based on role assignment

  Home page that allows user logins and redirects based on role
  Rahul Palani
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Drug Interaction Checker - Home</title>
    <link rel="stylesheet" href="css/styles.css">
</head>
<body>
<div class="container">
    <h1>Drug Interaction Checker</h1>
    <p>Log in or create an account to continue</p>

    <form id="loginForm">
        <label for="email">Email:</label>
        <input type="email" id="email" required>
        <label for="password">Password:</label>
```

```
      <input type="password" id="password" required>
      <button type="submit">Login</button>
    </form>

    <p>New User?</p>
    <button onclick="window.location.href='register.html'">Create Account</button>
</div>
<script src="js/supabase.js"></script>
<script src="js/home.js"></script>
</body>
</html>
```

patient.html

```
<!--
  patient.html     Virginia Tech      Sprint 1,2,3
  Patient portal, this file handles all patient-related web UI structure.

  Allows patients to view their assigned doctor, see their prescriptions, and submit questionnaires.
  Also includes medication safety warning logic based on FDA data.
  Rahul Palani
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Patient Portal</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
<div class="topbar">
  <div class="role" id="headerRole"></div>
  <button class="tab" onclick="window.location.href='patient.html'">Dashboard</button>
  <button class="tab" onclick="window.location.href='reminders.html'">Reminders</button>
  <button id="logoutBtn" class="logout-btn">Logout</button>
</div>
<div class="page">
```

```
<div class="panel">
  <div id="doctorInfo">Not assigned</div>
</div>

<!-- Medications Panel -->
<div class="panel">
  <h3>Your Medications</h3>
  <ul id="rxList" class="list"></ul>
</div>

<!-- Medication Survey Panel (new) -->
  <!-- Sprint 3, Eitan Pupkin -->
<div class="panel" id="medSurveyPanel" style="display:none;">
  <h3>Medication Survey</h3>
  <div id="medSurveyMedicationLabel" class="muted"></div>

  <label for="surveyAdherence">Are you taking this as prescribed? (1–5)</label>
  <input type="number" id="surveyAdherence" min="1" max="5">

  <label for="surveyEffectiveness">How well is it working for you? (1–5)</label>
  <input type="number" id="surveyEffectiveness" min="1" max="5">

  <label for="surveySideEffects">Any side effects?</label>
  <input type="text" id="surveySideEffects">
```

```
  <button id="surveySaveBtn" type="button">Save Survey</button>
</div>
<!-- Sprint 3, Steven An -->
<!-- Questionnaire Panel -->
<div class="panel">
  <h3>Health Questionnaire</h3>
  <form id="questionnaireForm">
    <label>Do you have any allergies?</label>
    <input type="text" id="q_allergies">

    <label>Current symptoms:</label>
    <input type="text" id="q_symptoms">

    <label>Are you currently taking any OTC medications?</label>
    <input type="text" id="q_otc">

    <button type="submit">Submit Questionnaire</button>
  </form>
</div>

<!-- Sprint 3, Steven An -->
<!-- Medication Safety Warnings Panel -->
<div class="panel">
  <h3>Medication Safety Warnings</h3>
  <div id="safetyWarnings" class="muted">
```

We’ll show important safety notices here based on your current prescriptions

<script src="js/supabase.js"></script>

<script src="js/logo.js"></script>

<script src="js/patient.js"></script>

</body>

</html>

register.html

```
<!--
  register.html   Virginia Tech       Sprint 1,2,3
  Registration page for creating new user accounts

  Allows new users to create an account with role selection
  Everett Miceli
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Create Account</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
<div class="container logo-offset">
  <h2>Create Account</h2>
  <form id="registerForm">
    <label for="name">Full Name:</label>
    <input type="text" id="name" required>

    <label for="email">Email:</label>
    <input type="email" id="email" required>
```

```
    <label for="userId">User ID (unique):</label>
    <input type="text" id="userId" required>

    <label for="phoneNumber">Phone Number (Optional):</label>
    <input type="tel" id="phoneNumber" placeholder="555-555-5555">

    <label for="password">Password:</label>
    <input type="password" id="password" required>

    <label for="role">Role:</label>
    <select id="role">
      <option value="patient">Patient</option>
      <option value="doctor">Doctor</option>
    </select>

    <button type="submit">Register</button>
  </form>
  <p>Already have an account?</p>
  <button onclick="window.location.href='home.html'">Back to Login</button>
</div>

<script src="js/supabase.js"></script>
<script src="js/logo.js"></script>
<script src="js/register.js"></script>
```

```
</body>
</html>
```

reminders.html

```
<!--
  reminders.html   Virginia Tech      Sprint 3
  Rahul Palani & Everett Miceli
  Patient reminders page
-->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Reminders</title>
  <link rel="stylesheet" href="css/styles.css">
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>
<body>
<div class="topbar">
  <div class="role" id="headerRole"></div>
  <button class="tab" onclick="window.location.href='patient.html'">Dashboard</button>
  <button class="tab active"
onclick="window.location.href='reminders.html'">Reminders</button>
  <button id="logoutBtn" class="logout-btn">Logout</button>
  <!-- ^^ Top bar -->

</div>

<div class="page">
```

```
<div class="panel">
  <h3>Add Reminder</h3>
  <form id="reminderForm">
    <label>Medication</label>
    <input type="text" id="rem_medication" placeholder="Metformin 500mg" required>

    <label>Frequency</label>
    <select id="rem_frequency">
      <option value="once_daily">Once Daily</option>
      <option value="twice_daily">Twice Daily</option>
      <option value="three_times_daily">Three Times Daily</option>
      <option value="every_8_hours">Every 8 Hours</option>
      <option value="weekly">Weekly</option>
    </select>

    <label>Times (comma-separated, 24hr)</label>
    <input type="text" id="rem_times" placeholder="08:00, 20:00">

    <label>With Meal?</label>
    <select id="rem_with_meal">
      <option value="false">No</option>
      <option value="true">Yes</option>
    </select>

    <label>Notes</label>
```

```
    <input type="text" id="rem_notes" placeholder="Optional">

    <button type="button" id="setReminderBtn">Set reminder</button>
   </form>
  </div>

  <div class="panel" id="listPanel">
   <h3>Your Reminders</h3>
   <ul id="reminderList" class="list"></ul>
  </div>
</div>

<script src="js/supabase.js"></script>
<script src="js/logo.js"></script> <!-- loads logo/home button icon in top left-->
<script src="js/reminders.js"></script>
</body>
</html>
```